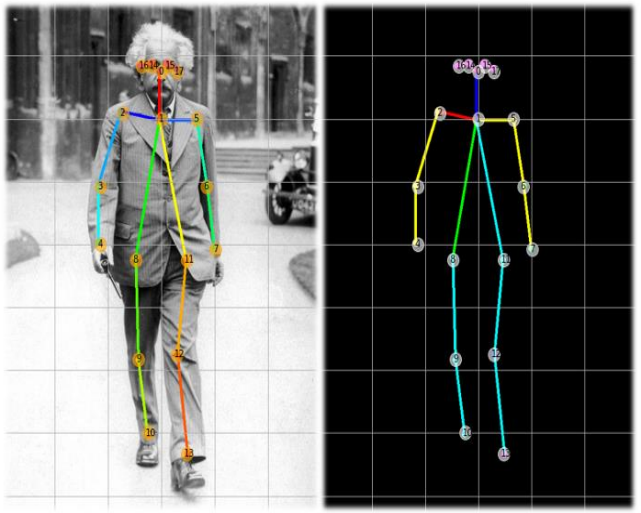
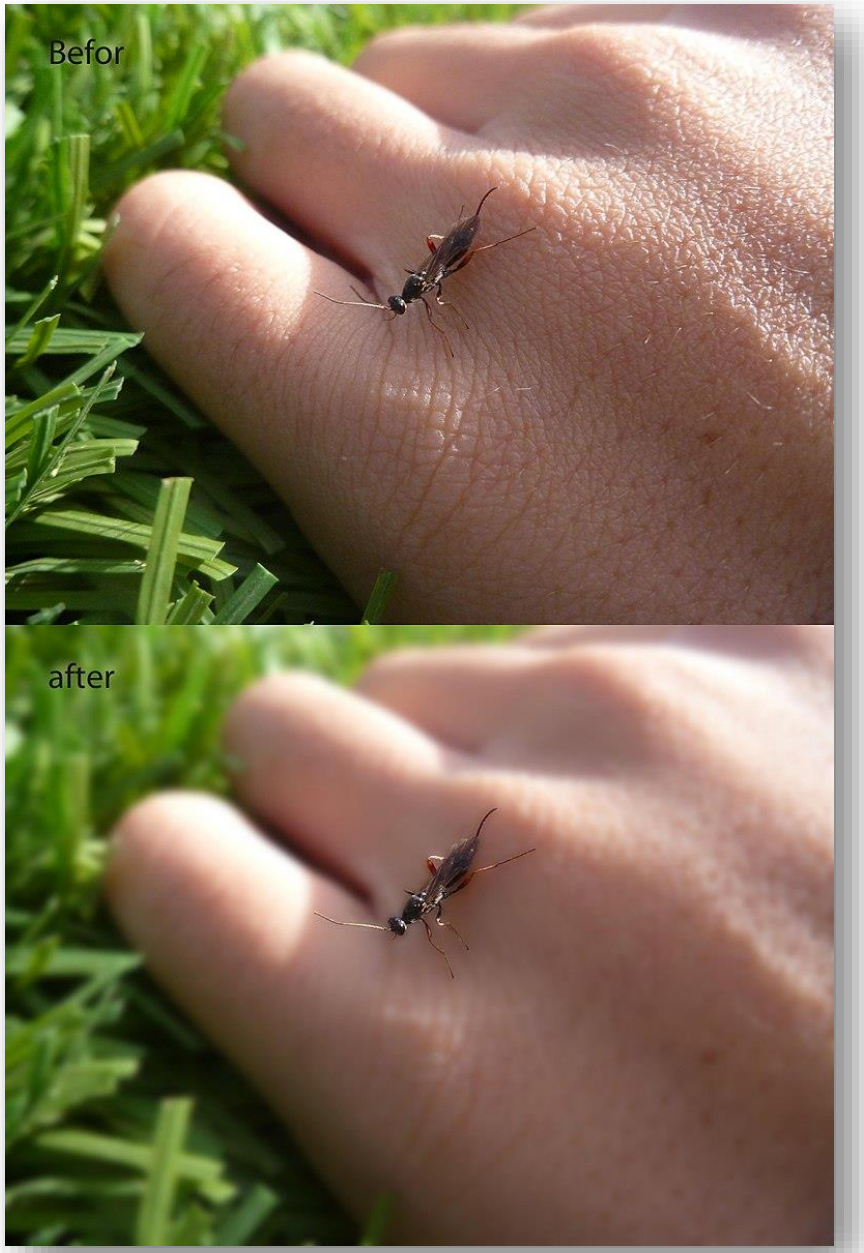
DIGITAL IMAGE PROCESSING

การใช้การประมวลผลทางคอมพิวเตอร์ (Computer Algorithm) ในการจัดการรูปภาพ มีจุดประสงค์การทำงานที่หลากหลาย เช่น การปรับปรุงรูปภาพ (Enhancement) การตรวจจับวัตถุ (Detection) การแบ่งแยกบริเวณในภาพ (Segmentation) เป็นต้น

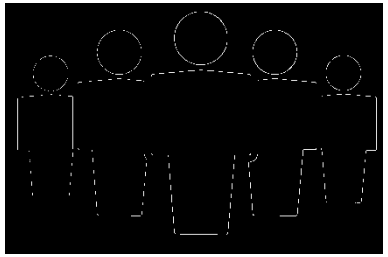








IMAGE



IMAGE



IMAGE

5 OBJECTS

HEIGHT 1.5 M

DESCRIPTOR

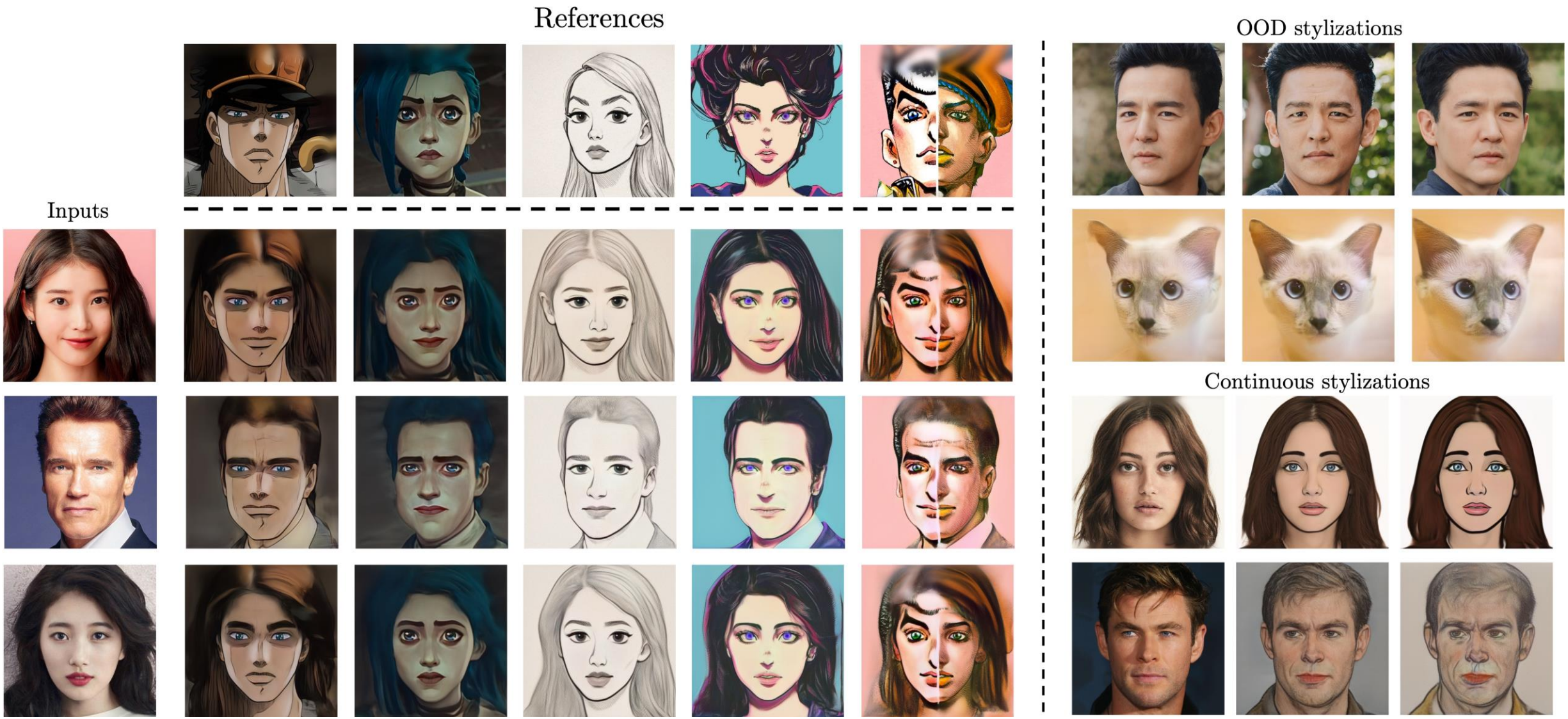
OUTPUT



IMAGE

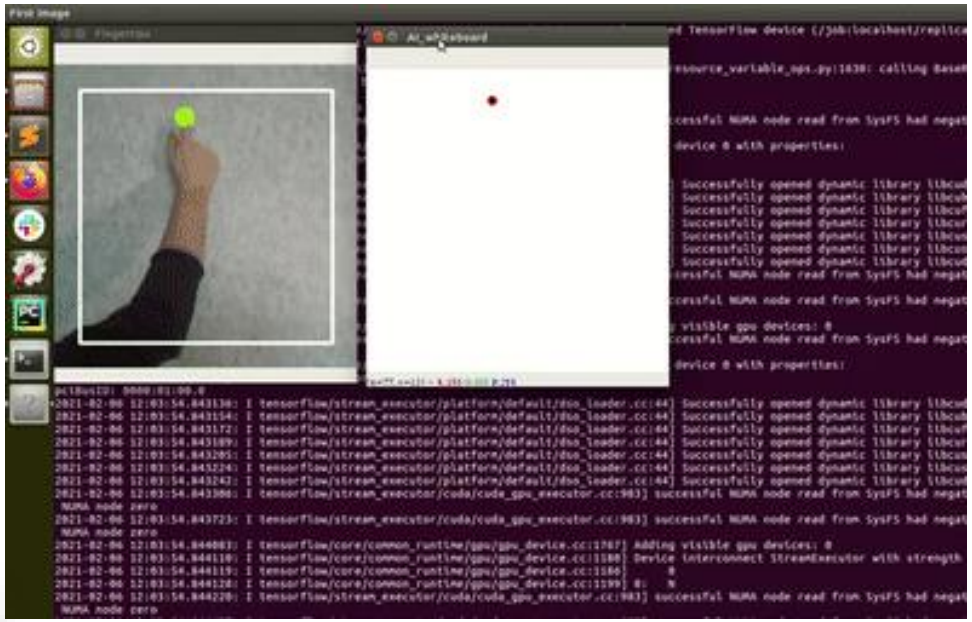
A cat that is lying down and looking at the camera

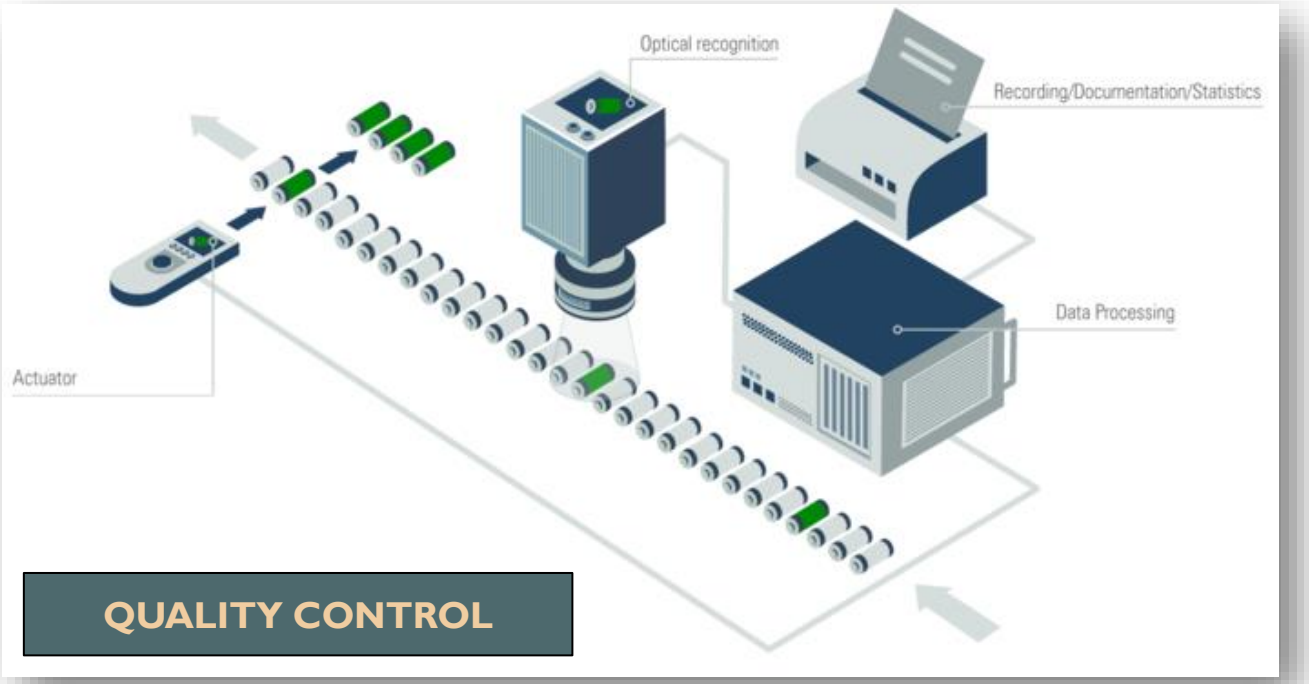
RECOGNITION / CLASSIFICATION


References
OOD stylizations
Inputs
Continuous stylizations



LICENSE PLATE RECOGNITION





QUALITY CONTROL

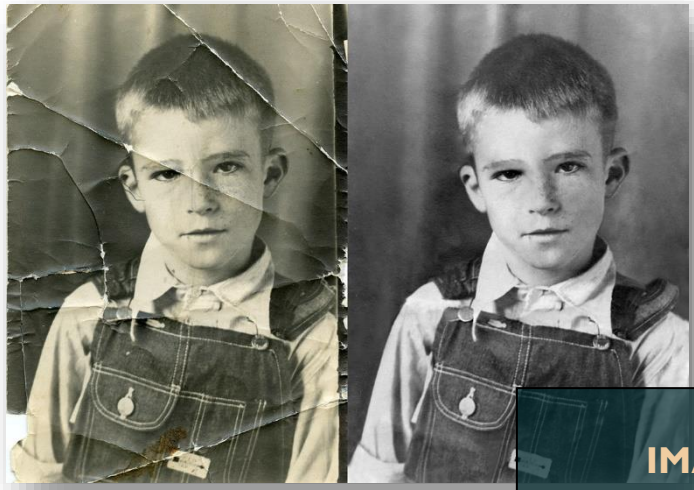
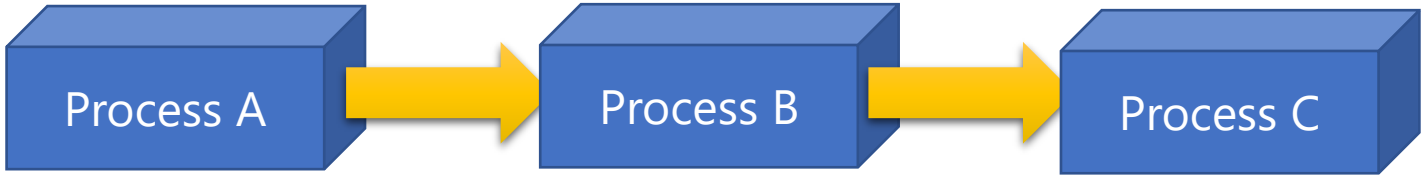
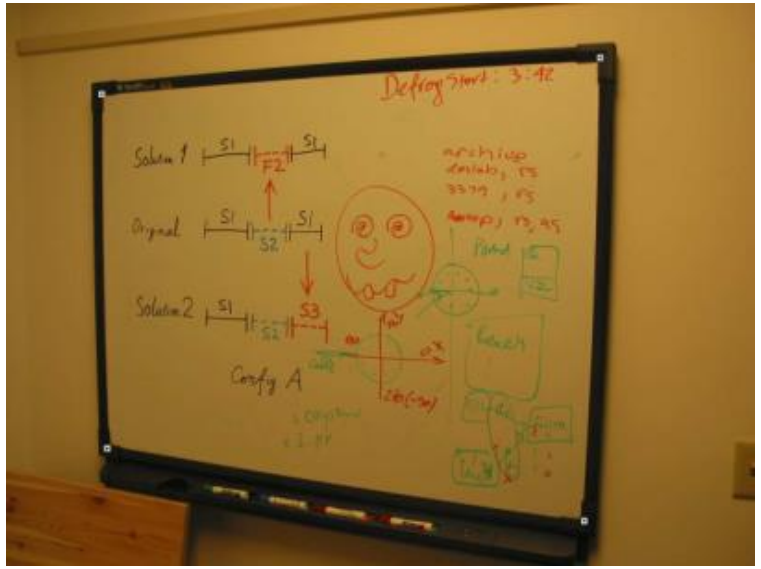


IMAGE RESTORATION

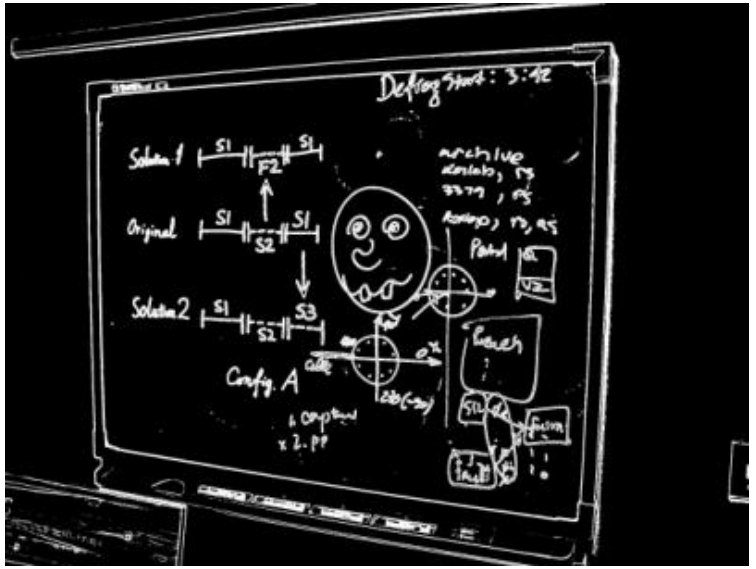
*** ถ้าโจทย์ปัญหายาก ให้ใช้วิธีกำหนดเงื่อนไข, ขอบเขตของงาน, แบ่งเป็นงานย่อย







TAKE IMAGE FROM CAMERA



EDGE & CORNER DETECTION

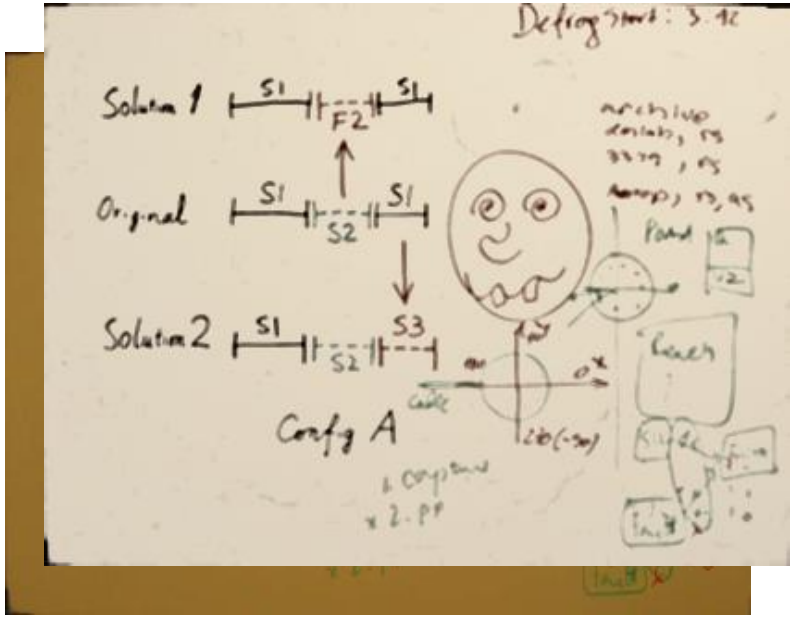


IMAGE TRANSFORMATION

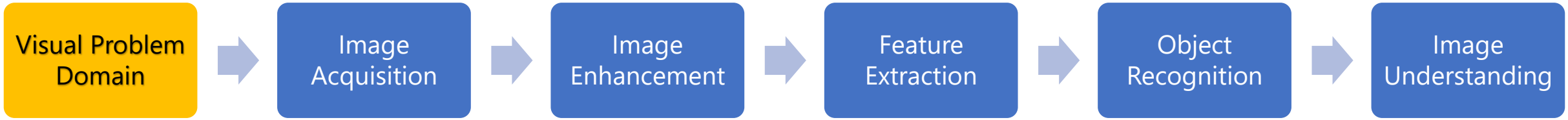
ตัวอย่างการใช้เครื่องมือ IMAGE PROCESSING ที่จะได้เรียนใน CLASS นี้

Class Projects

การวัดผลสัมฤทธิ์ในการเรียน	ร้อยละ
1. การบ้านและโจทย์ปัญหาในชั้นเรียน	30
2. โครงงานด้านการประมวลภาพ (Web App)	40
3. สอบปลายภาค	30

โครงงานแบ่งการออกแบบระบบการประมวลผลภาพเป็นส่วนย่อย

1. การเก็บข้อมูลภาพ (Image Acquisition)
2. การตรวจสอบคุณภาพและปรับปรุงคุณภาพของภาพ (Quality Assessment & Enhancement)
3. การตรวจจับบริเวณของวัตถุที่ต้องการ (Segmentation)
4. การสกัดคุณลักษณะสำคัญ (Feature Extraction) → คัดแยก (Classification) / วิเคราะห์ (Analysis)
5. การวัดประสิทธิภาพการทำงานของโครงงาน (Evaluation)



แนวทางการหาโจทย์ปัญหา งานทางด้านการประมวลผลภาพ (อย่างง่าย)

1.) งานที่ใช้ตา (มอง) และทำการตัดสินใจ

2.) งานที่มีการทำงานอย่างต่อเนื่อง => ต้องการระบบอัตโนมัติ

3.) งานที่จำเป็นต้องมีการแก้ไข ปรับปรุงภาพ

การคัดแยกวัตถุ

การตรวจสอบความสมบูรณ์

การหาวัตถุที่กำหนดในภาพ

การนับชิ้นวัตถุ

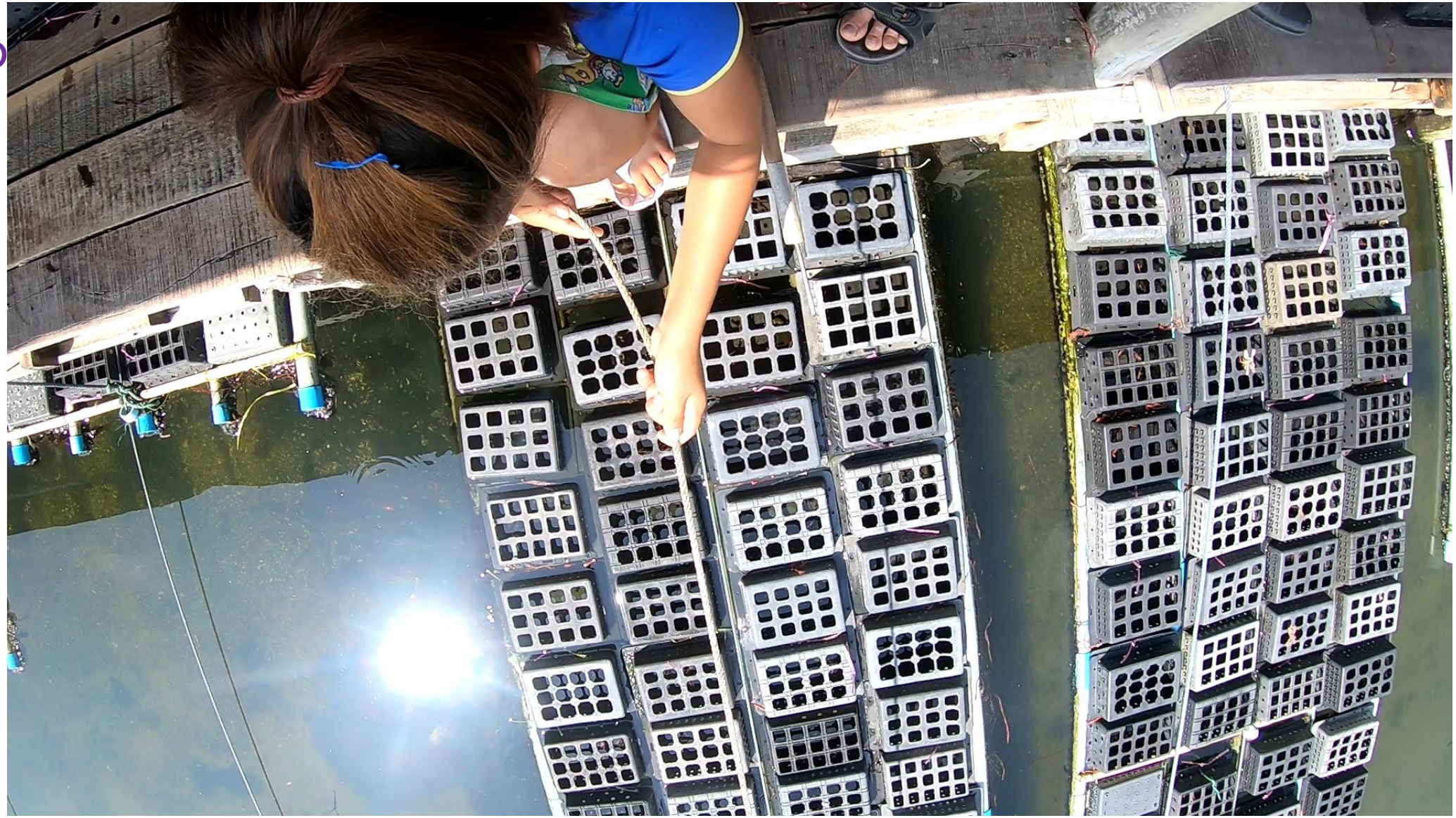
การวัดปริมาณจากสิ่งที่เห็น

Proble



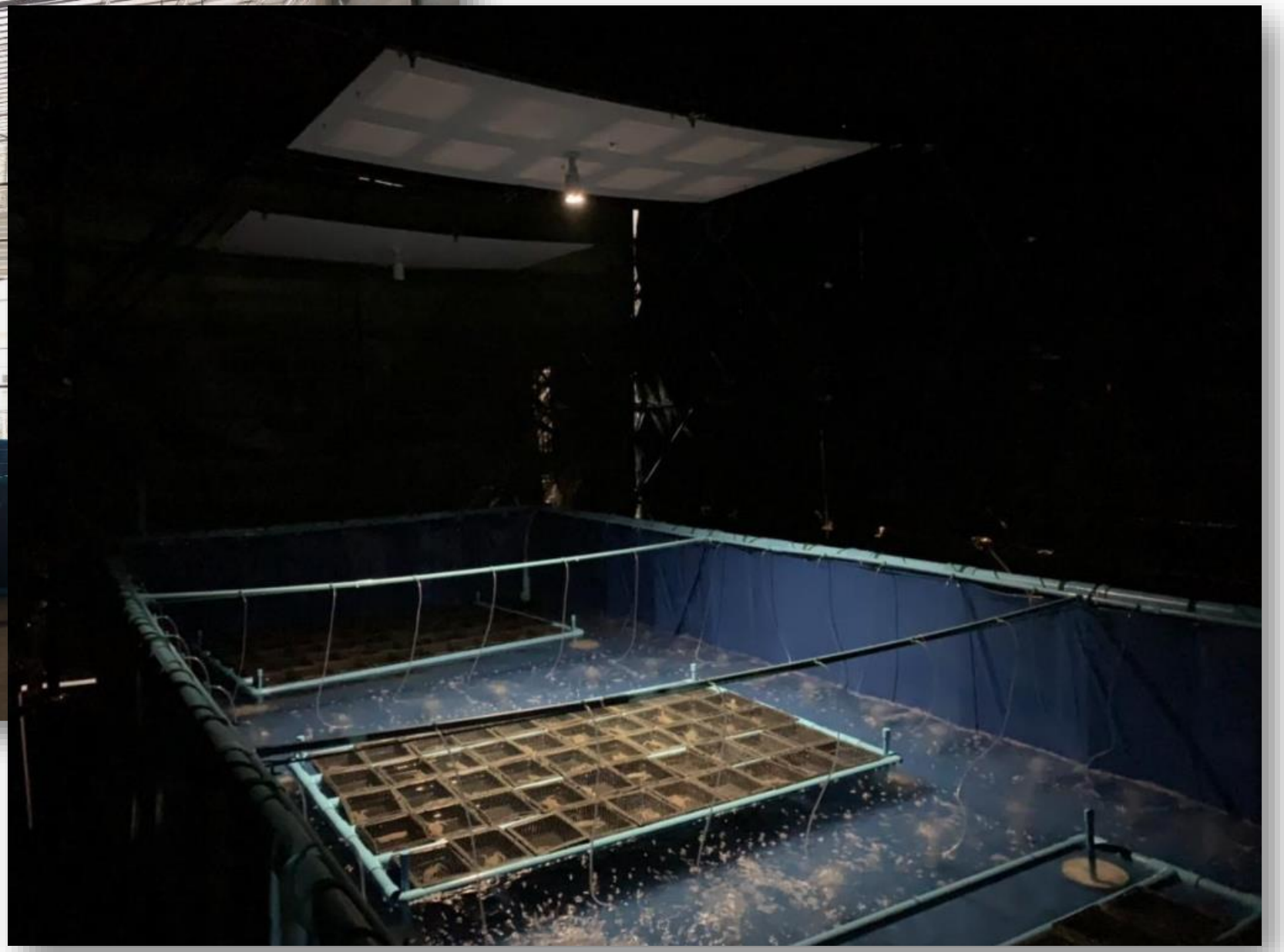
24 x 7 Workload

Prob



skilled worker (need to train)

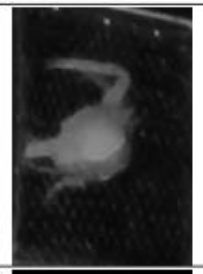
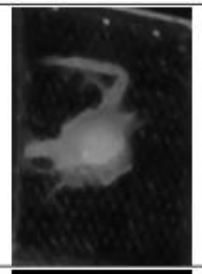
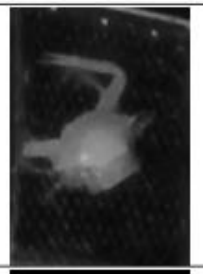
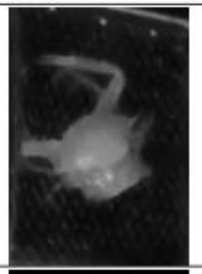
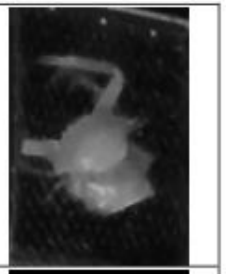


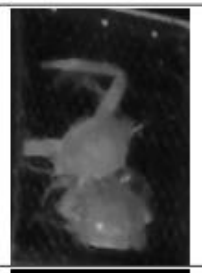
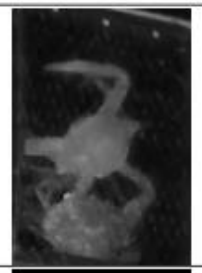
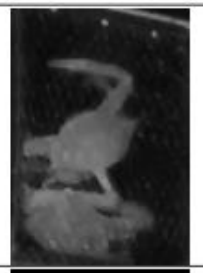
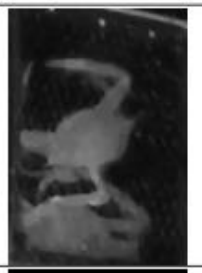
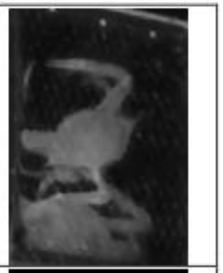



A8.txt
A8_[0].png
A8_[1].png
A8_[2].png
A8_[3].png
A8_[4].png
A8_[5].png
A9.txt
A9_[0].png
A9_[1].png
A9_[2].png
A9_[3].png
A9_[4].png
A9_[5].png
B0.txt
B0_[0].png
B0_[1].png
B0_[2].png
B0_[3].png
B0_[4].png
B0_[5].png
B1.txt
B1_[0].png
B1_[1].png
B1_[2].png
B1_[3].png
B1_[4].png
B1_[5].png

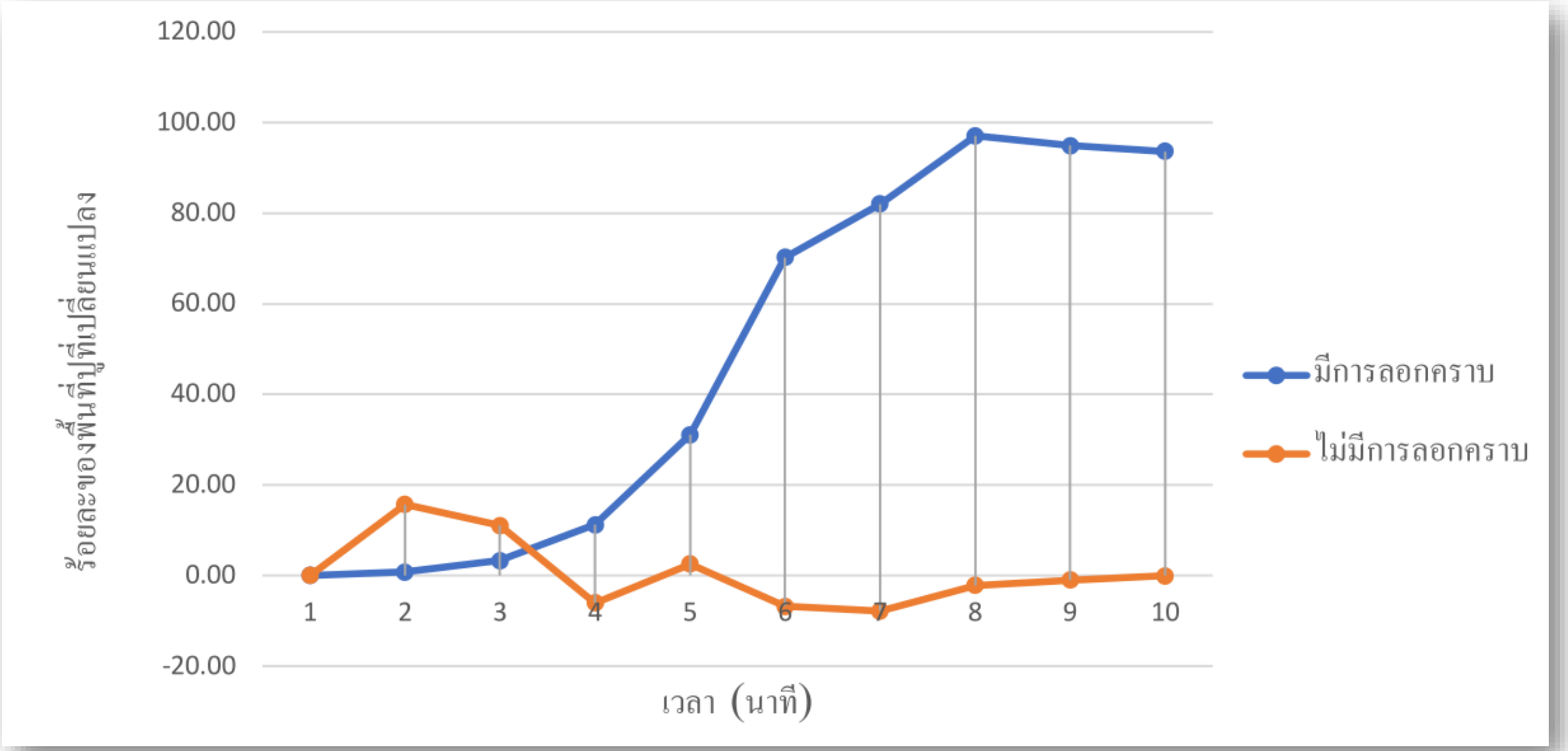
Algorithm

Experiment

ภาพกล้อง					
ภาพตรวจจับ					
เวลา (นาที)	1	2	3		
พื้นที่ (ร้อยละ)	15.02	15.14	15.51		

ภาพกล้อง					
ภาพตรวจจับ					
เวลา (นาที)	6	7	8	9	10
พื้นที่ (ร้อยละ)	25.57	27.34	29.60	29.28	29.09

Conclusion


120.00
100.00
80.00
60.00
40.00
20.00
0.00
-20.00
1
2
3
4
5
6
7
8
9
10
ร้อยละของพื้นที่ที่เปลี่ยนแปลง
เวลา (นาที)
มีการลอกคราบ
ไม่มีการลอกคราบ



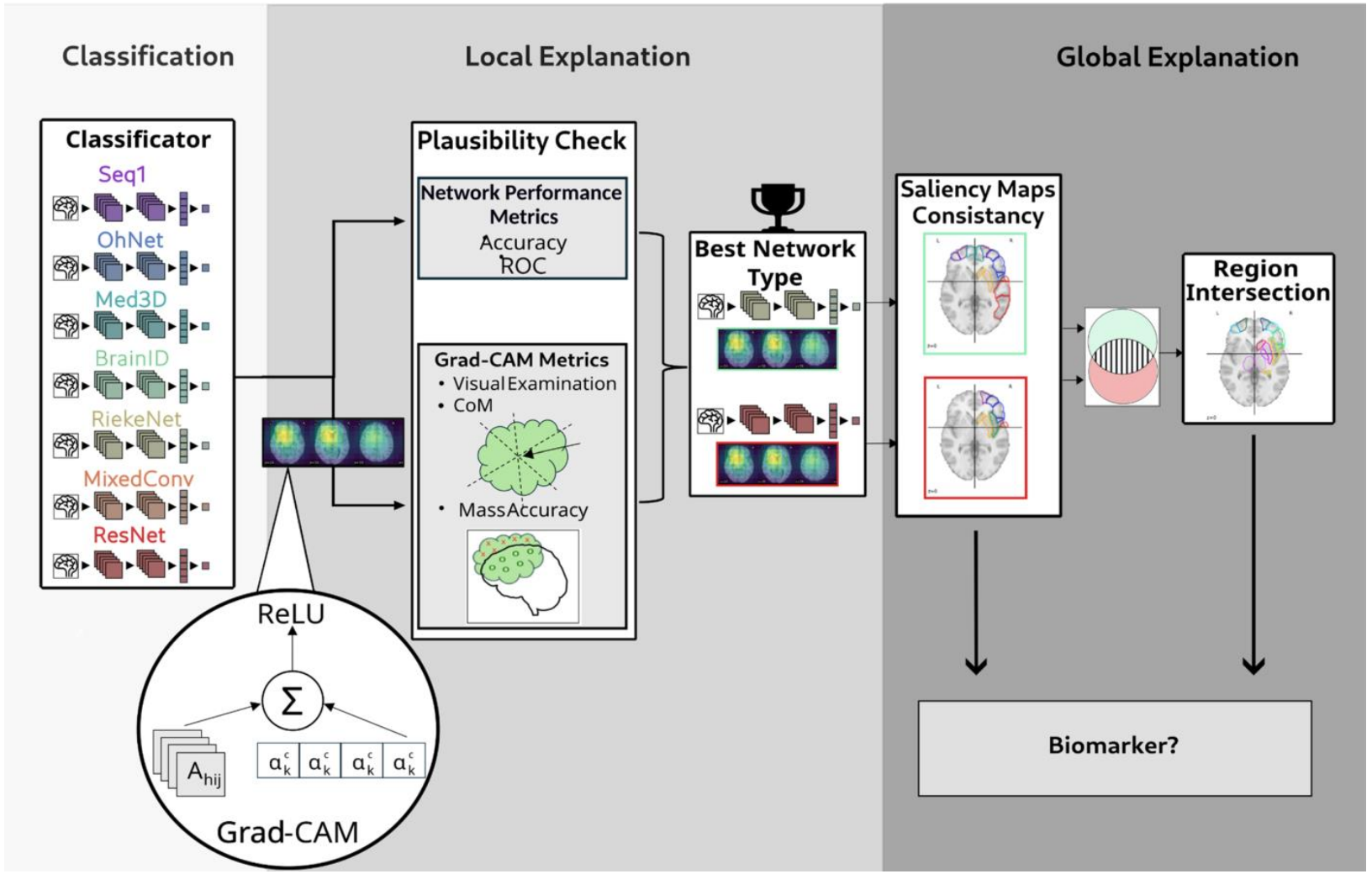


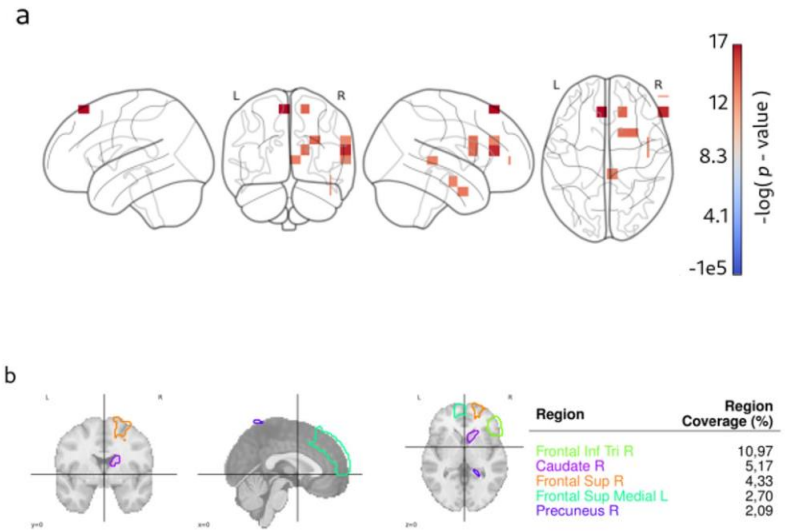


Research in Image Processing

Image Processing (310-3311)

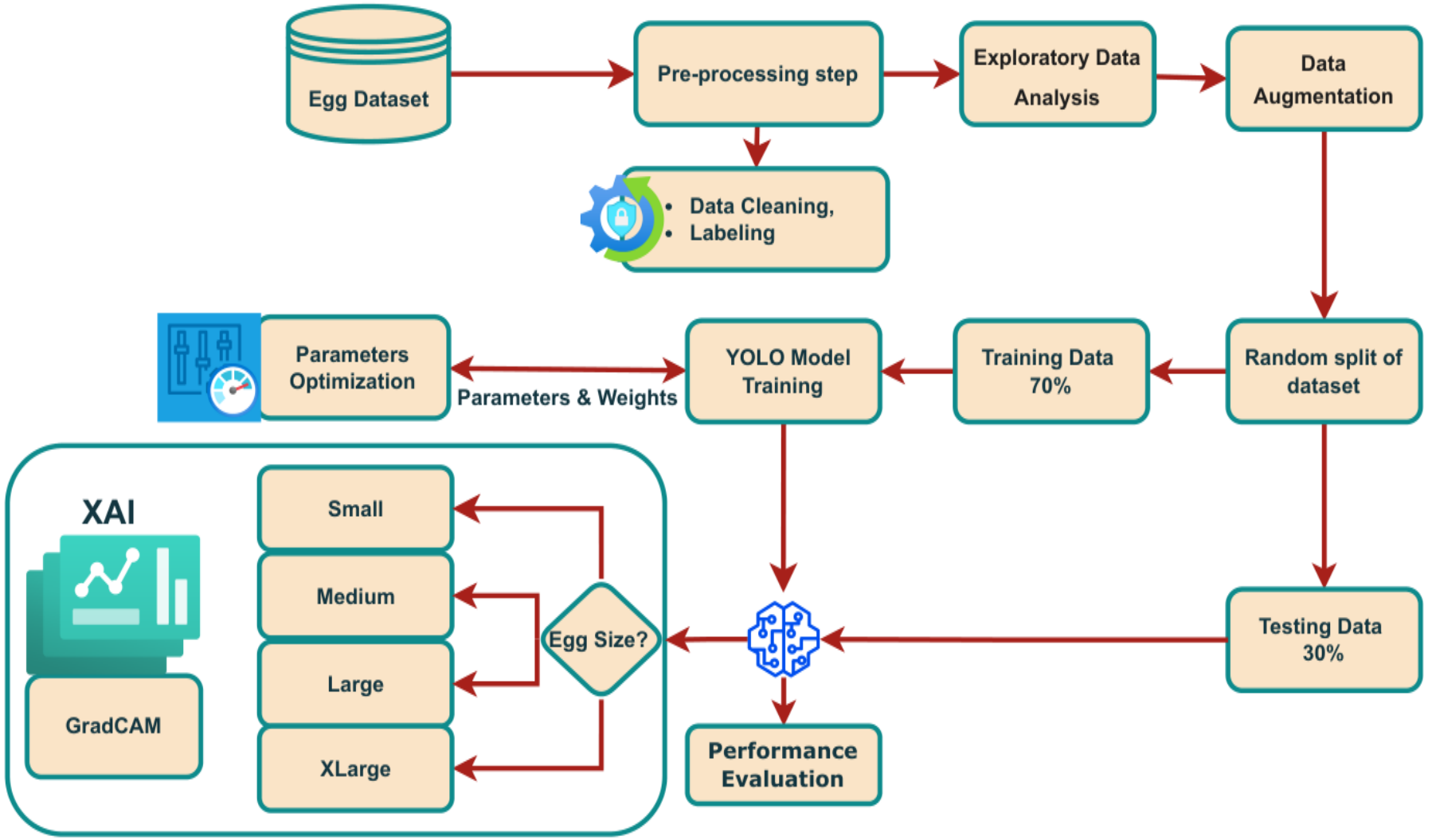
Decision processes in 3D structural MRI schizophrenia classification evaluated with saliency maps [2026]



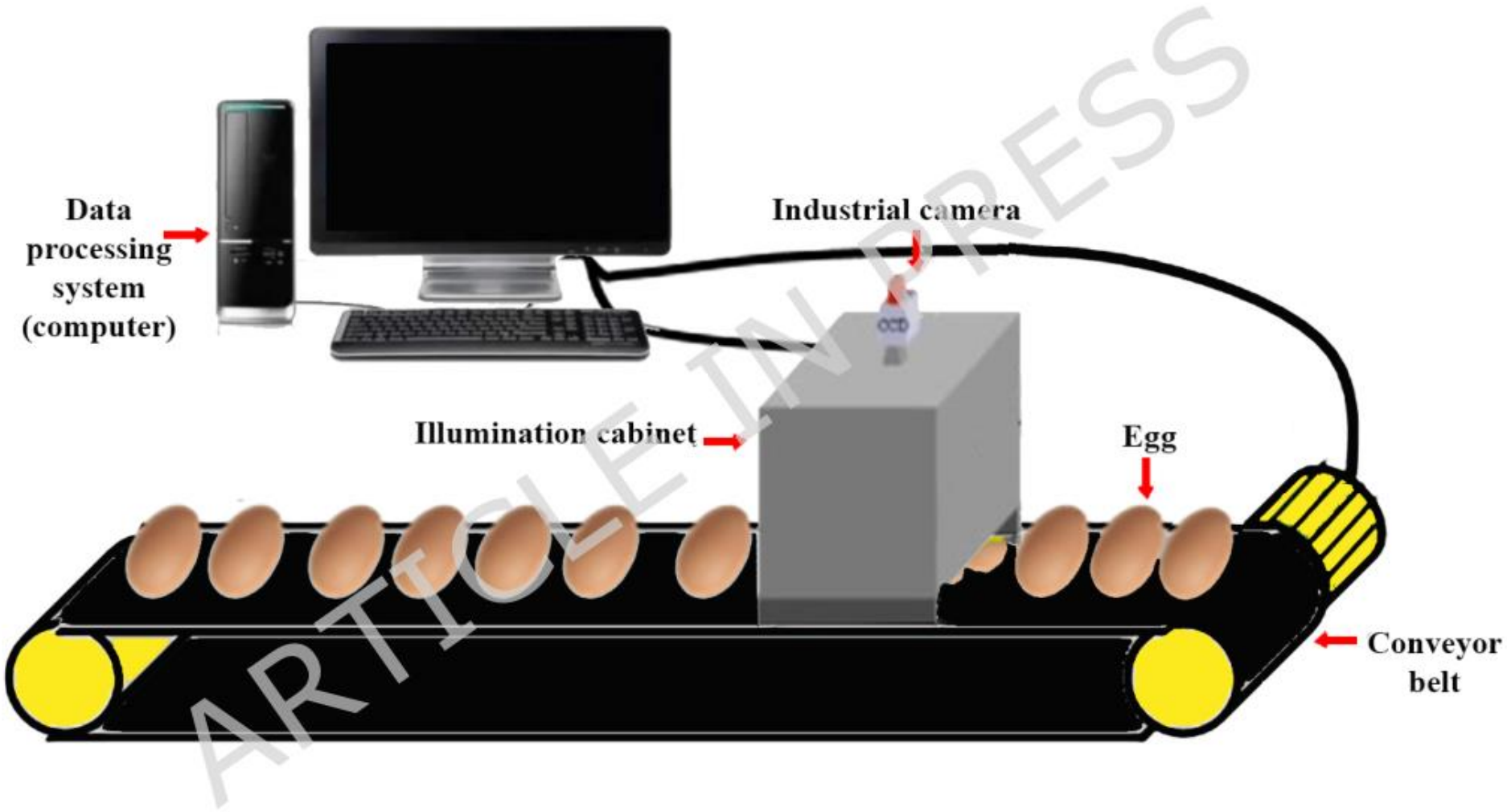


	Schizophrenia	Control
Number of subjects	101	91
Age (years) mean ± STD	34.2 ± 11.2	32.3 ± 11.8
Gender (male/female)	78/23	61/30

Explainable YOLO architectures for egg size measurement and classification [2026]

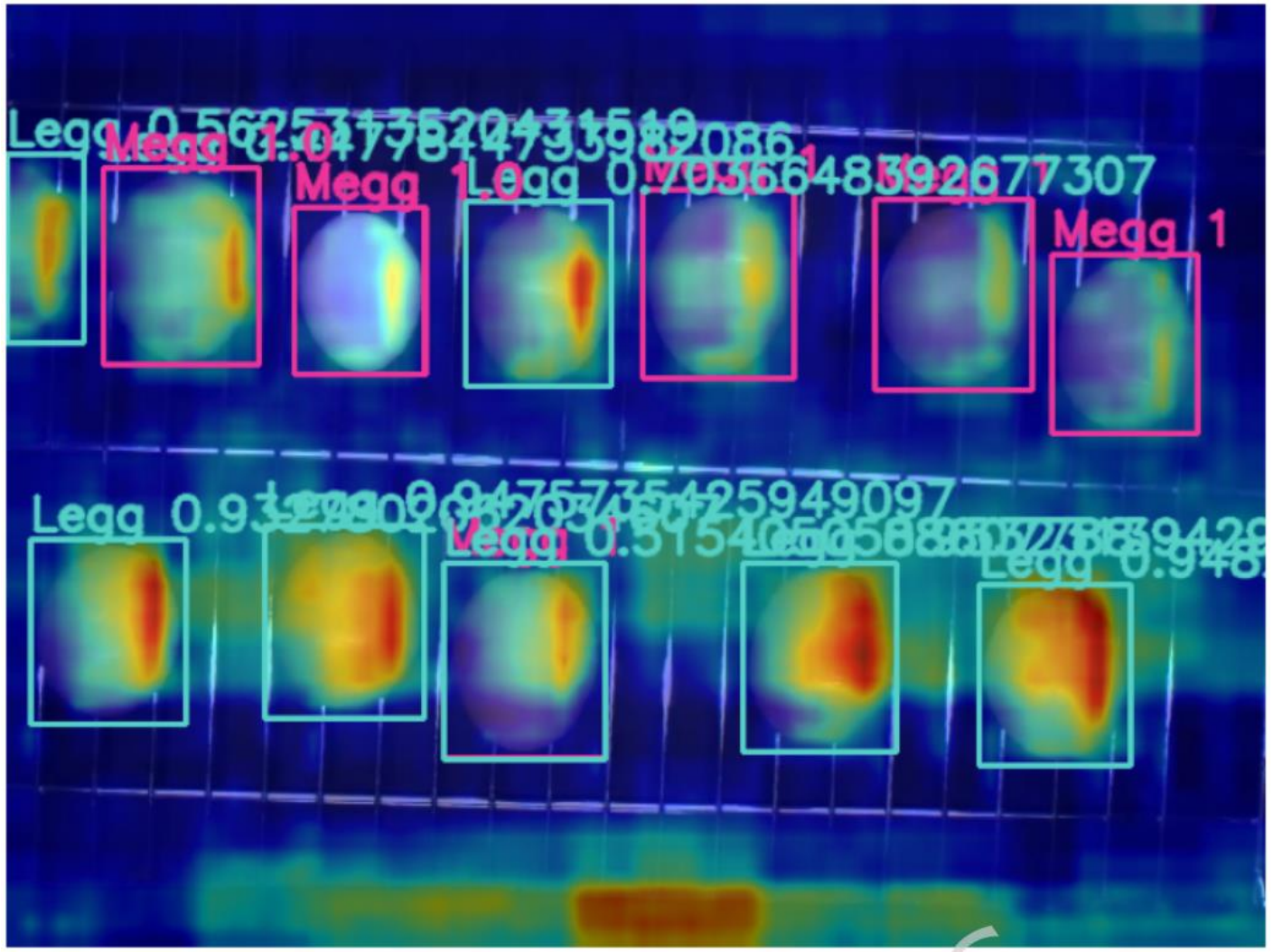


Explainable YOLO architectures for egg size measurement and classification [2026]



ARTICLE IN PRESS

Explainable YOLO architectures for egg size measurement and classification [2026]



Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization

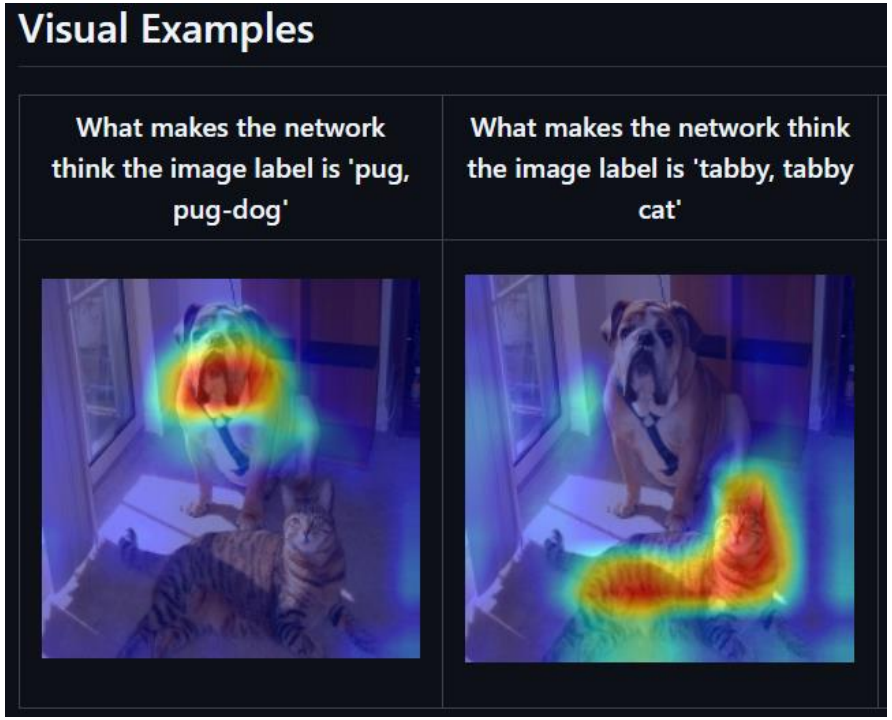












Image Generation with A.I.

It is a very popular tool nowadays.

How it work? How to use it?





Why do you need to know?

AI image generators use machine learning algorithms to create images based on textual descriptions. Users can input a variety of **prompts**, and the AI will generate corresponding visuals, making these tools valuable for artists, designers, and content creators.

How it work ?

Apart from being a fascinating subject, some understanding of the inner mechanics will make you a better artist.

You can use the tool correctly to achieve results with higher precision.

"Teddy bears wearing suit discussing a business proposal around office table"

Stable Diffusion



Diffusion model

Stable Diffusion belongs to a class of deep learning models called **diffusion models**. They are generative models, meaning they are designed to generate new data like what they have seen in training. In the case of Stable Diffusion, the data are images.

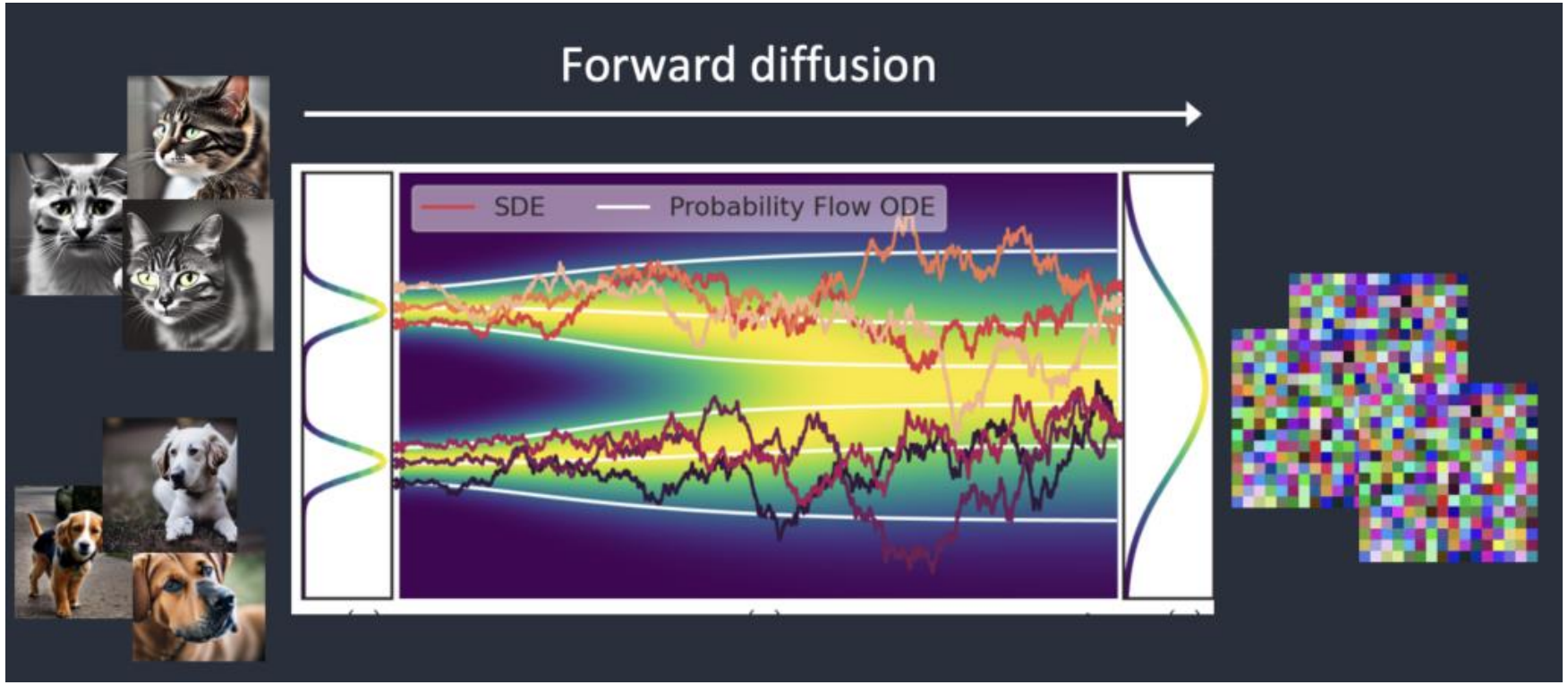
Why is it called the diffusion model?

Because its math looks very much like diffusion in physics.

Let’s go through the idea.

Let’s say I trained a diffusion model with only two kinds of images:

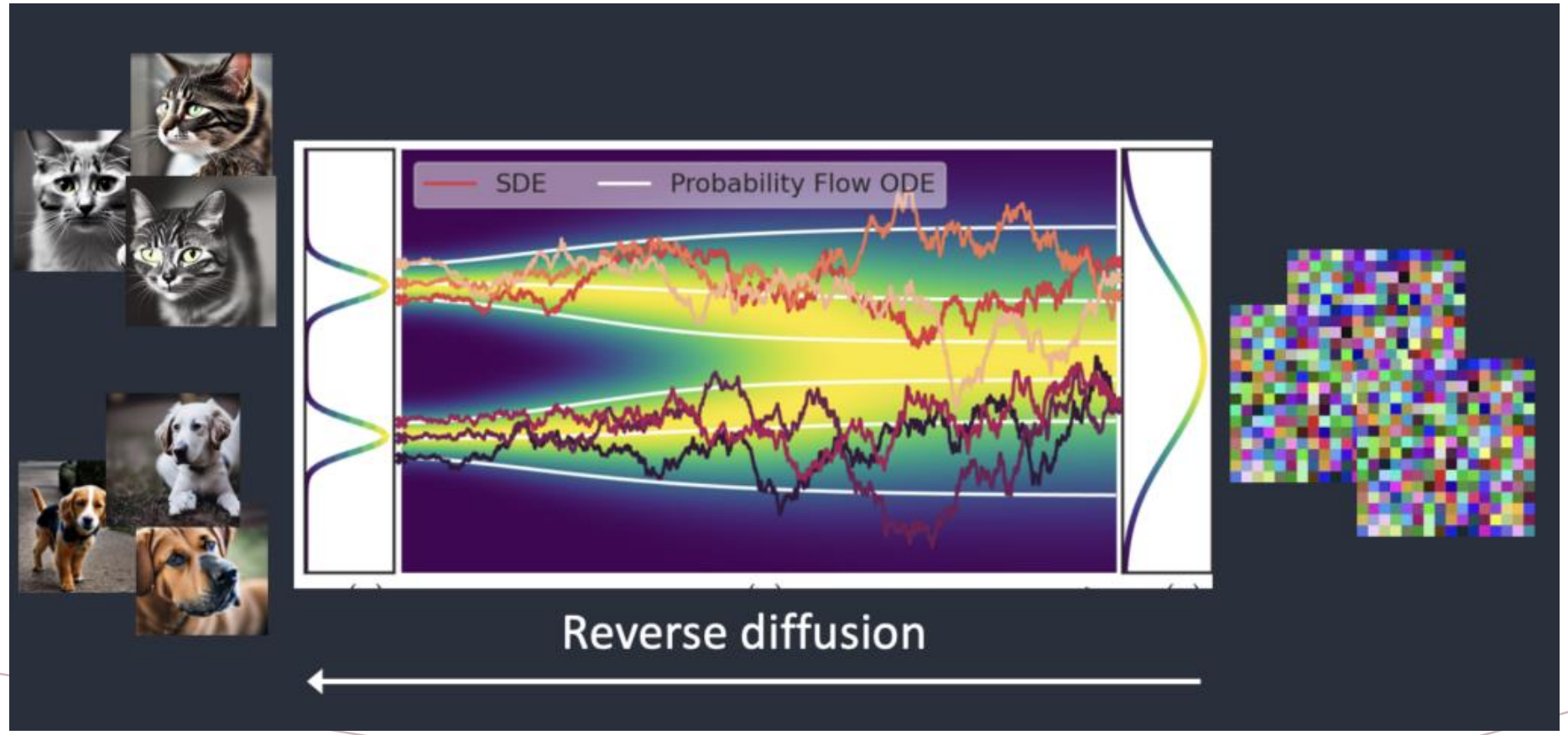
cats and dogs. In the figure below, the two peaks on the left represent the groups of cat and dog images. A **forward diffusion** process adds noise to a training image.



Forward diffusion



Now comes the exciting part. What if we can **reverse the diffusion**? Like playing a video backward. Going backward in time. We will see where the ink drop was initially added.



Technically, every diffusion process has two parts:

(1) **drift** and (2) **random motion**.

The reverse diffusion drifts towards either cat OR dog images but nothing in between.

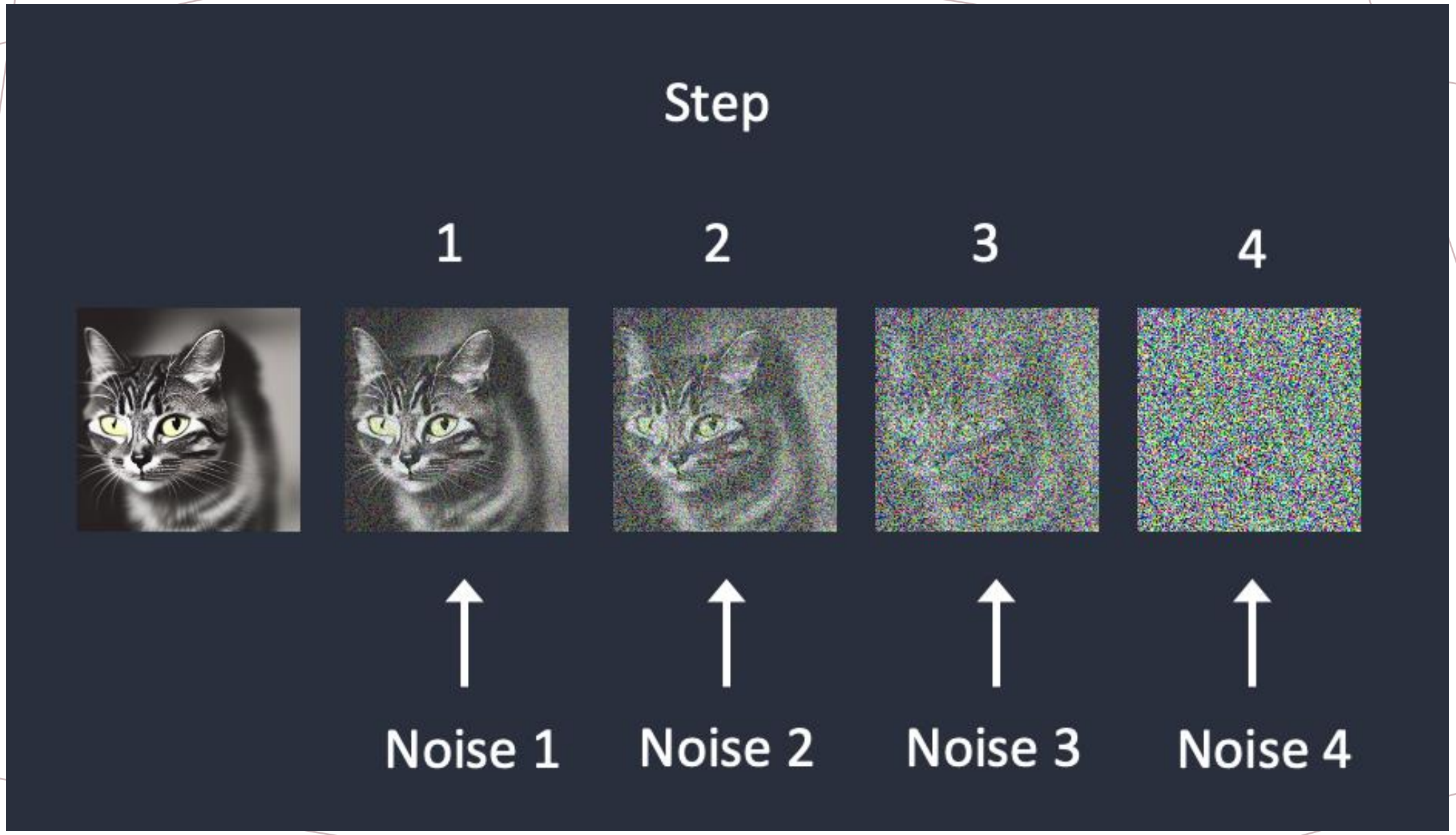
That’s why the result can either be a cat or a dog.

1. **Drift** (แนวโน้มทิศทางเฉพาะ)
2. **Random motion** (การเคลื่อนแบบสุ่ม)

How training is done

To reverse the diffusion, we need to know how much noise is added to an image. The answer is teaching a **neural network model to predict the noise added**. It is called the **noise predictor** in Stable Diffusion. It is a U-Net model.

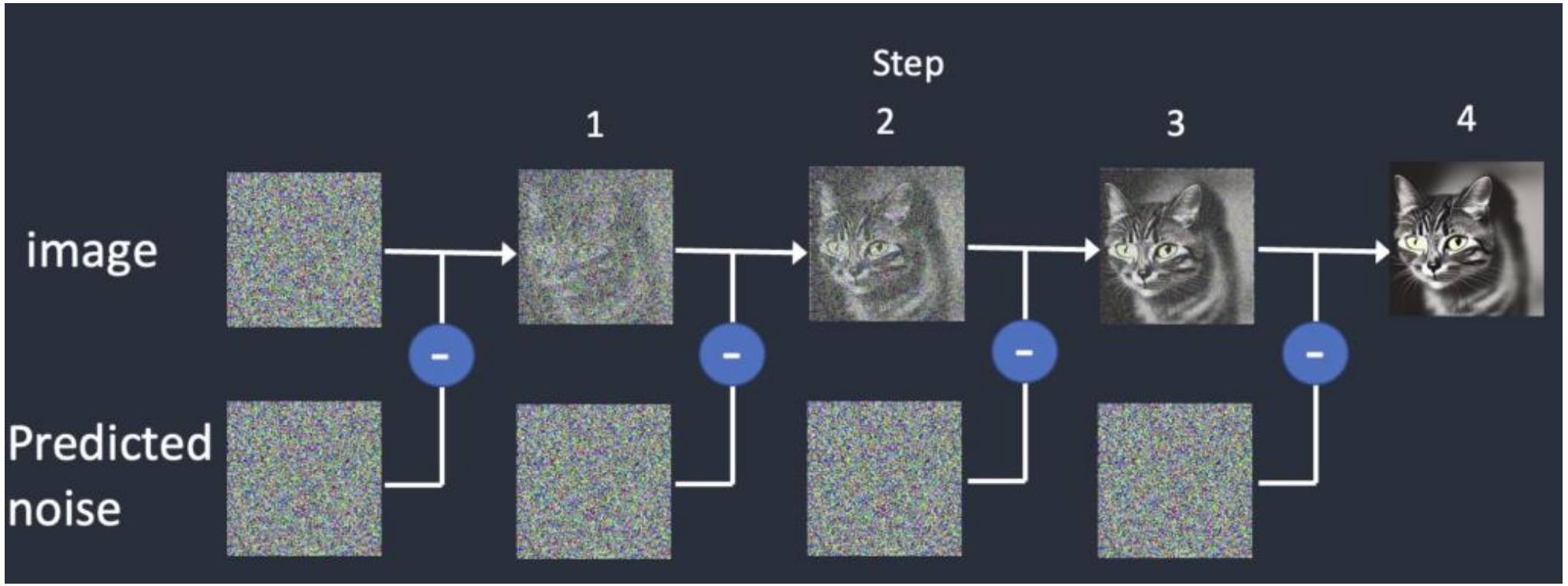
1. Pick a training image, like a photo of a cat.
2. Generate a random noise image.
3. Corrupt the training image by adding this noisy image up to a certain number of steps.
4. Teach the noise predictor to tell us how much noise was added. This is done by tuning its weights and showing it the correct answer.


Step
1
2
3
4
Noise 1
Noise 2
Noise 3
Noise 4

Reverse diffusion

Now we have the noise predictor. **How to use it?**

We first **generate a completely random image and ask the noise predictor to tell us the noise**. We then subtract this estimated noise from the original image. Repeat this process a few times. You will get an image of either a cat or a dog.



You may notice we have no control over generating a cat or dog's image. We will address this when we talk about conditioning. For now, image generation is **unconditioned**.

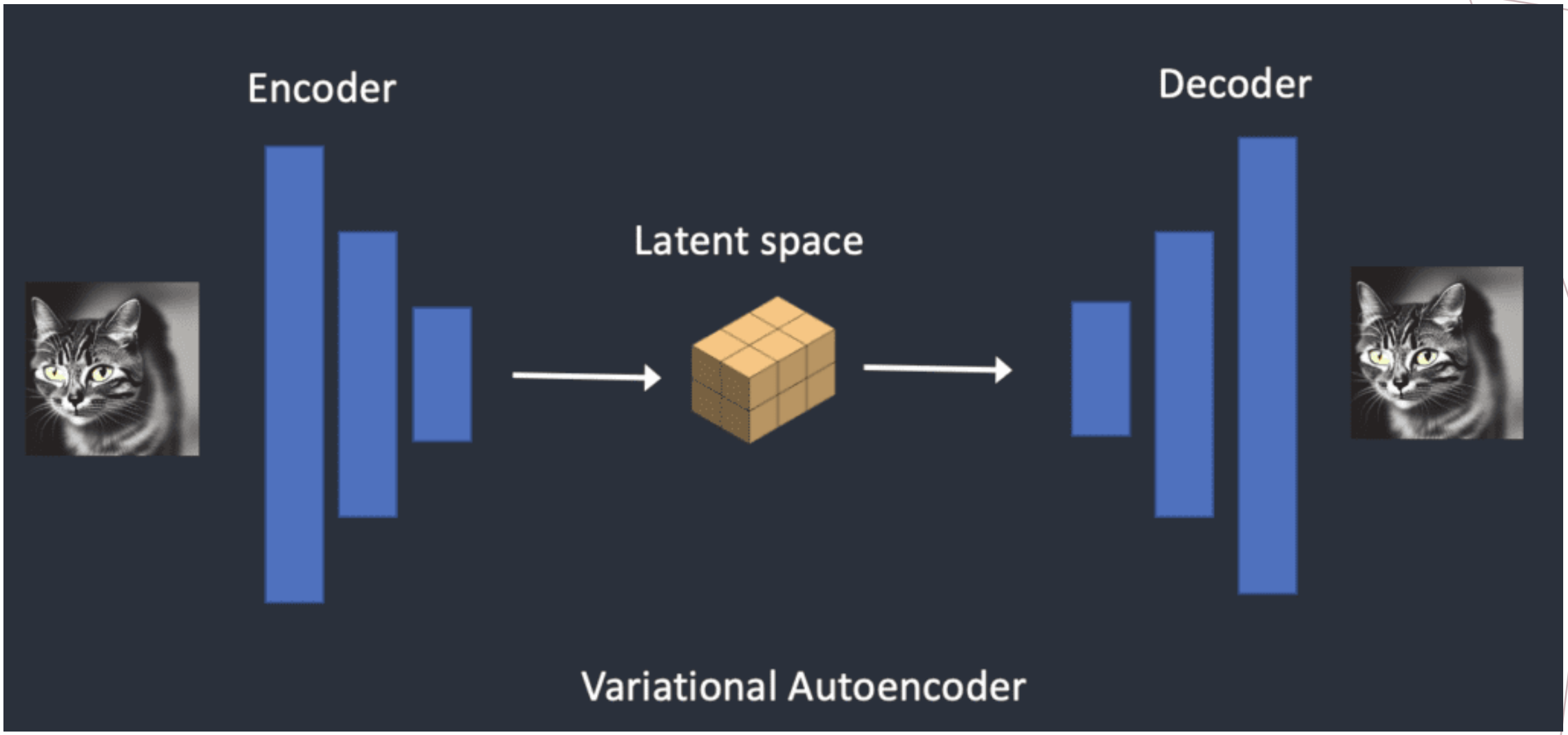
But doing this need a lot of GPU memory…

It is computationally very, **very slow.** You won't be able to run on *any* single GPU

Stable Diffusion is designed to solve the speed problem.

Stable Diffusion is a latent diffusion model.

Instead of operating in the high-dimensional image space, it first compresses the image into the **latent space**. The latent space is 48 times smaller, so it reaps the benefit of crunching a lot fewer numbers.


Encoder
Decoder
Latent space
Variational Autoencoder

Variational Autoencoder

It is done using a technique called the **variational autoencoder**. Yes, that's precisely what the VAE files are, but I will make it crystal clear later.

The **Variational Autoencoder (VAE)** neural network has two parts:

(1) an **encoder** (2) a **decoder**.

The encoder compresses an image to a lower dimensional representation in the latent space. The decoder restores the image from the latent space.

Reverse diffusion in latent space

1. A **random** latent space matrix **is generated**.
2. The **noise predictor estimates the noise** of the latent matrix.
3. The estimated noise is then **subtracted** from the latent matrix.
4. Steps 2 and 3 are **repeated** up to specific sampling steps.
5. The decoder of VAE converts the latent matrix to the final image.

Conditioning

Our understanding is incomplete: Where does the text prompt enter the picture?

This is where **conditioning** comes in. The purpose of conditioning is to **steer the noise predictor so that the predicted noise will give us what we want** after subtracting from the image.

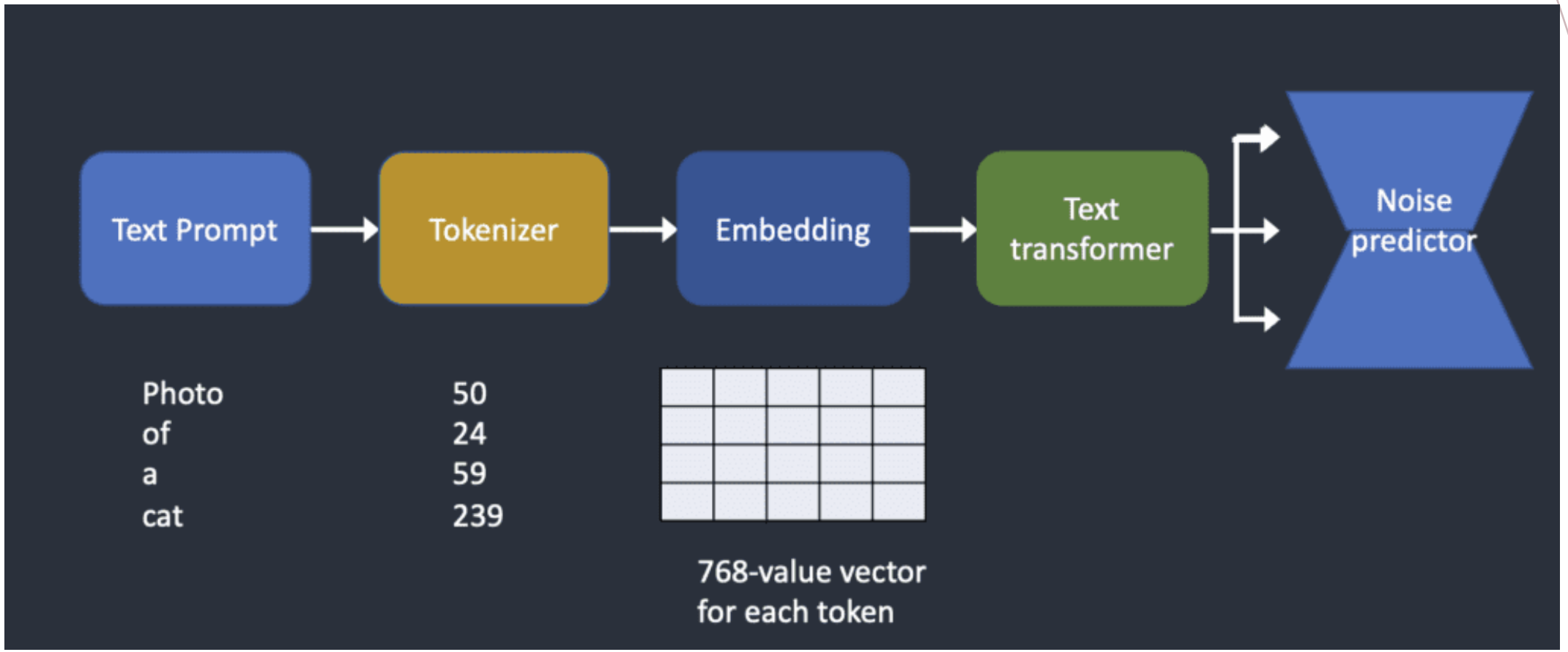
Text conditioning (text-to-image)

Our understanding is incomplete: Where does the text prompt enter the picture?

Tokenizer first converts each word in the prompt to a number called a **token**.

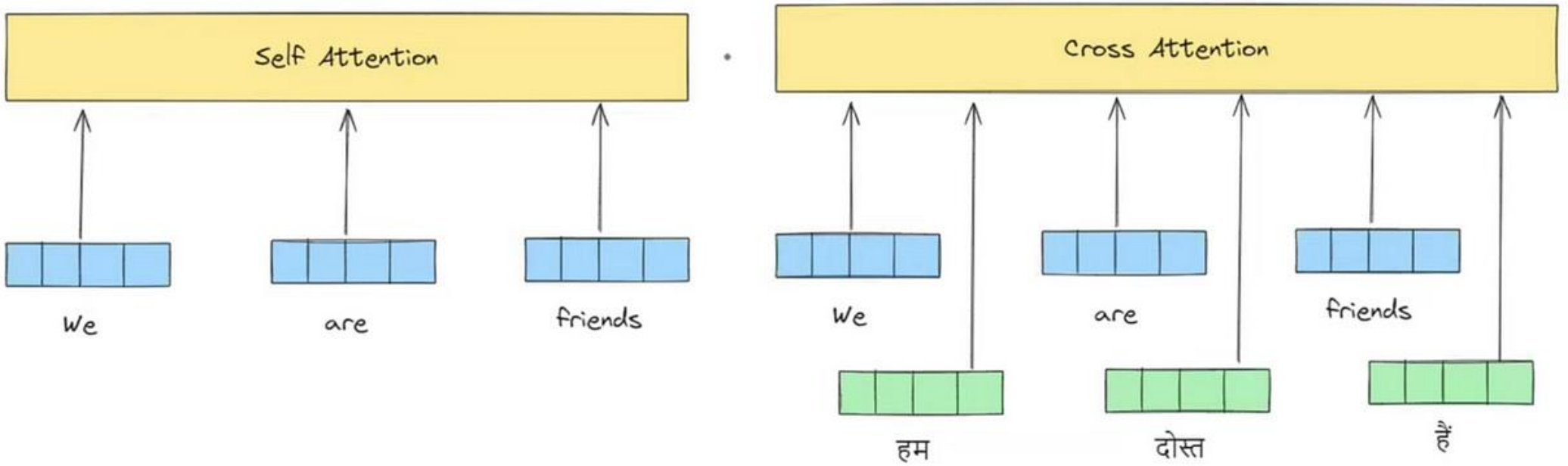
Each token is then converted to a 768-value vector called **embedding**.

The embeddings are then processed by the **text transformer** and are ready to be consumed by the noise predictor.

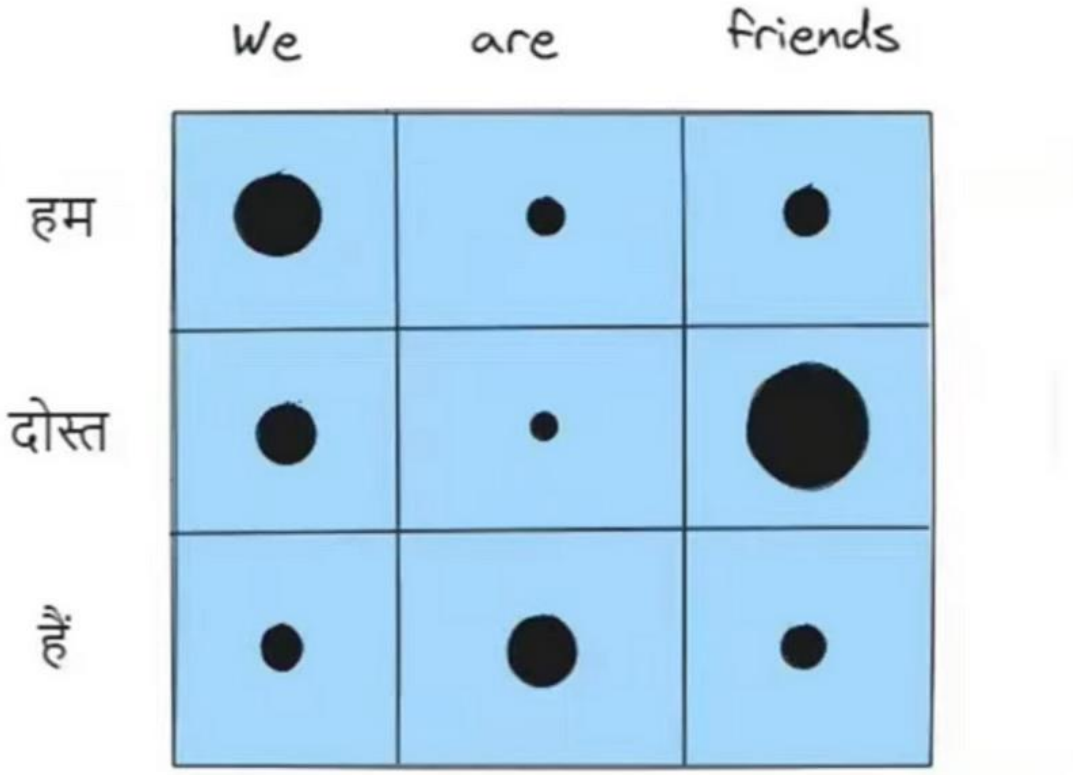

Text Prompt
Tokenizer
Embedding
Text transformer
Noise predictor
Photo
of
a
cat
50
24
59
239
768-value vector for each token

Cross-attention

The output of the text transformer is used **multiple times** by the noise predictor throughout the U-Net. The U-Net consumes it by a **cross-attention mechanism**. That's where the prompt meets the image.



Cross-attention



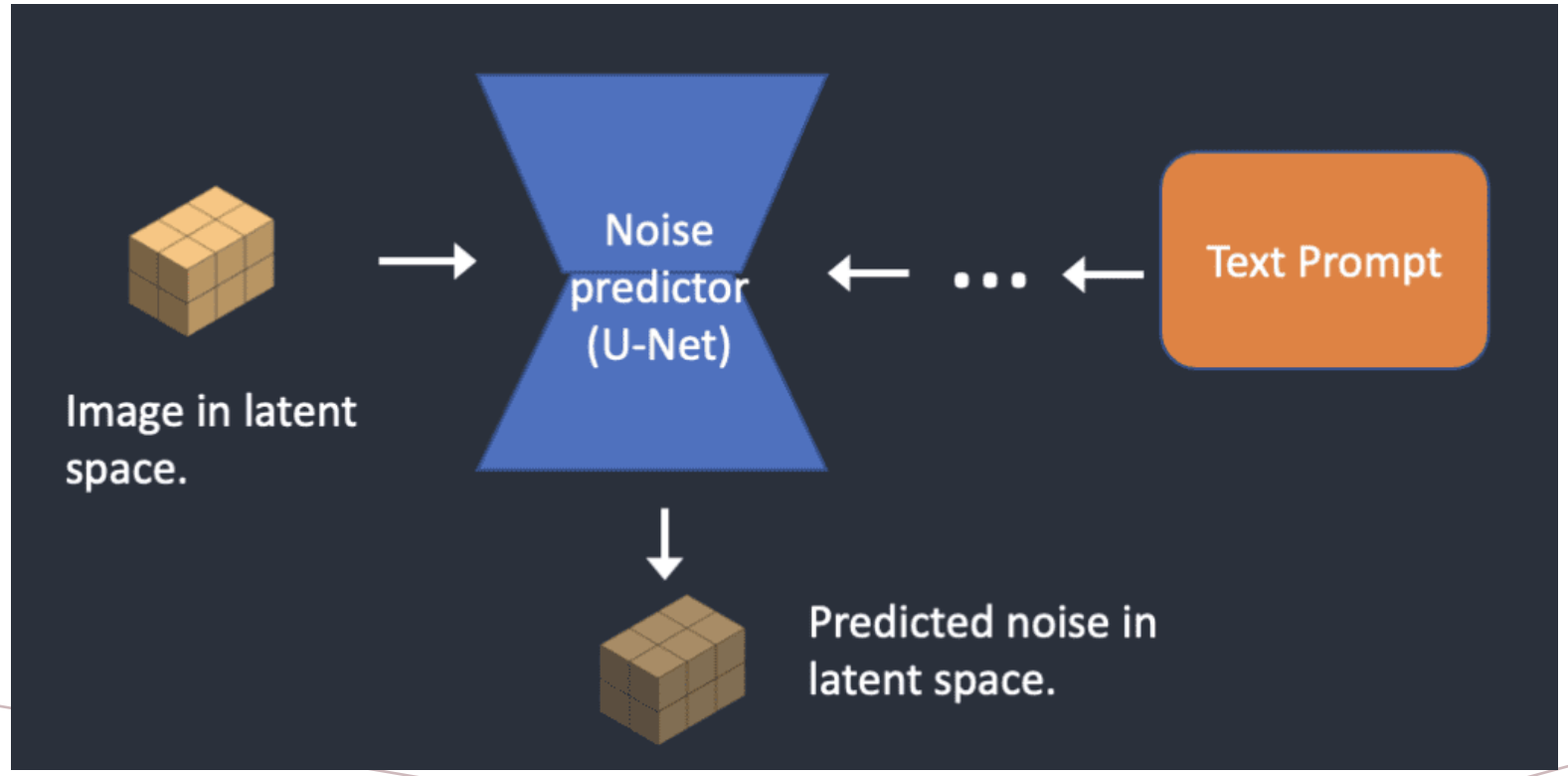
Text-to-image

Step 1. Stable Diffusion generates a random tensor in the latent space. You control this tensor by setting the seed of the random number generator.



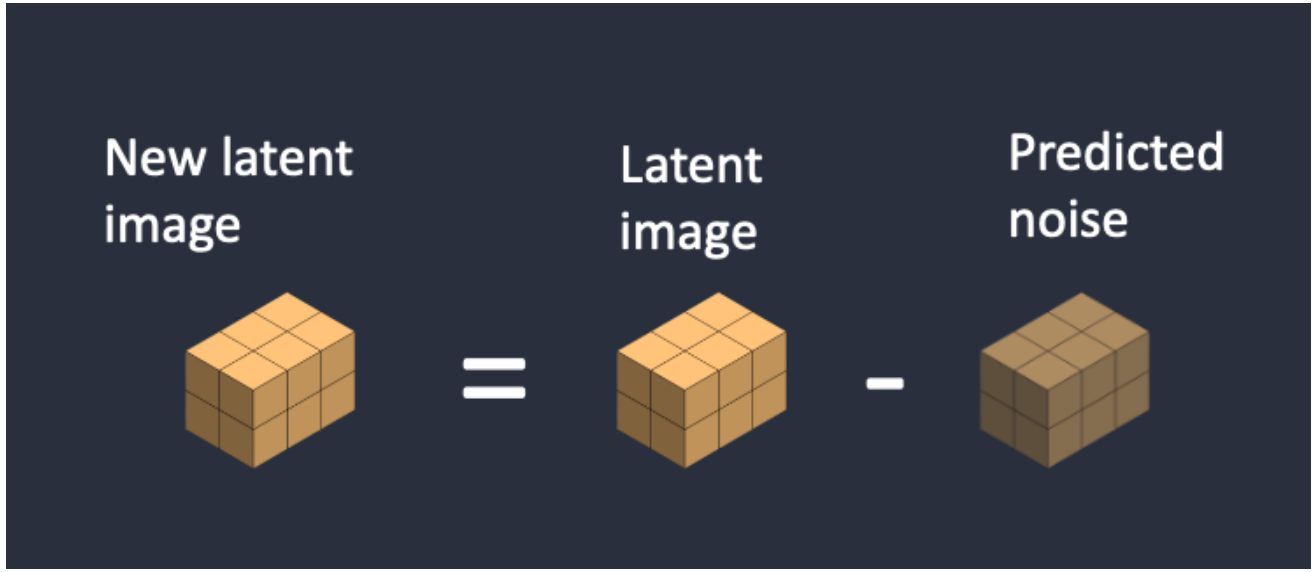
Text-to-image

Step 2. The noise predictor U-Net takes the latent noisy image and text prompt as input and **predicts** the noise, also in latent space (a 4x64x64 tensor).



Text-to-image

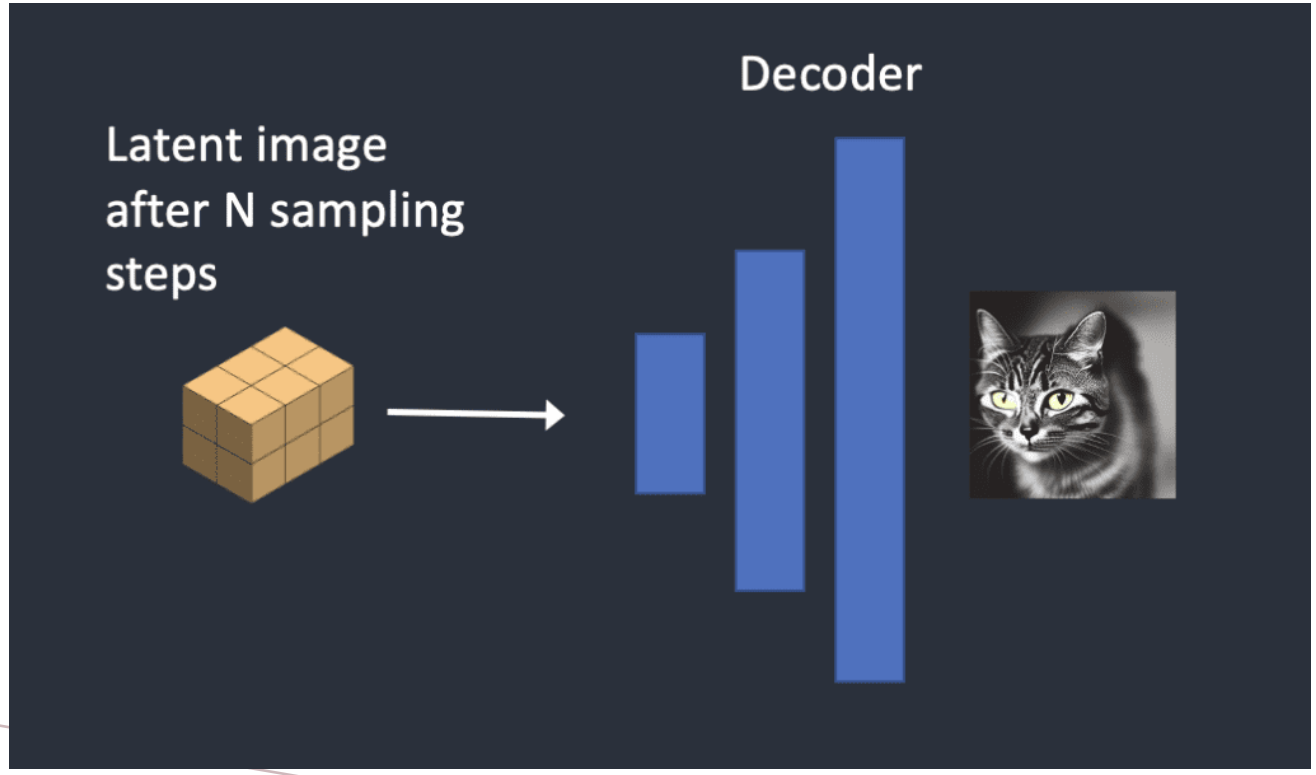
Step 3. Subtract the latent noise from the latent image. This becomes your **new latent image**.

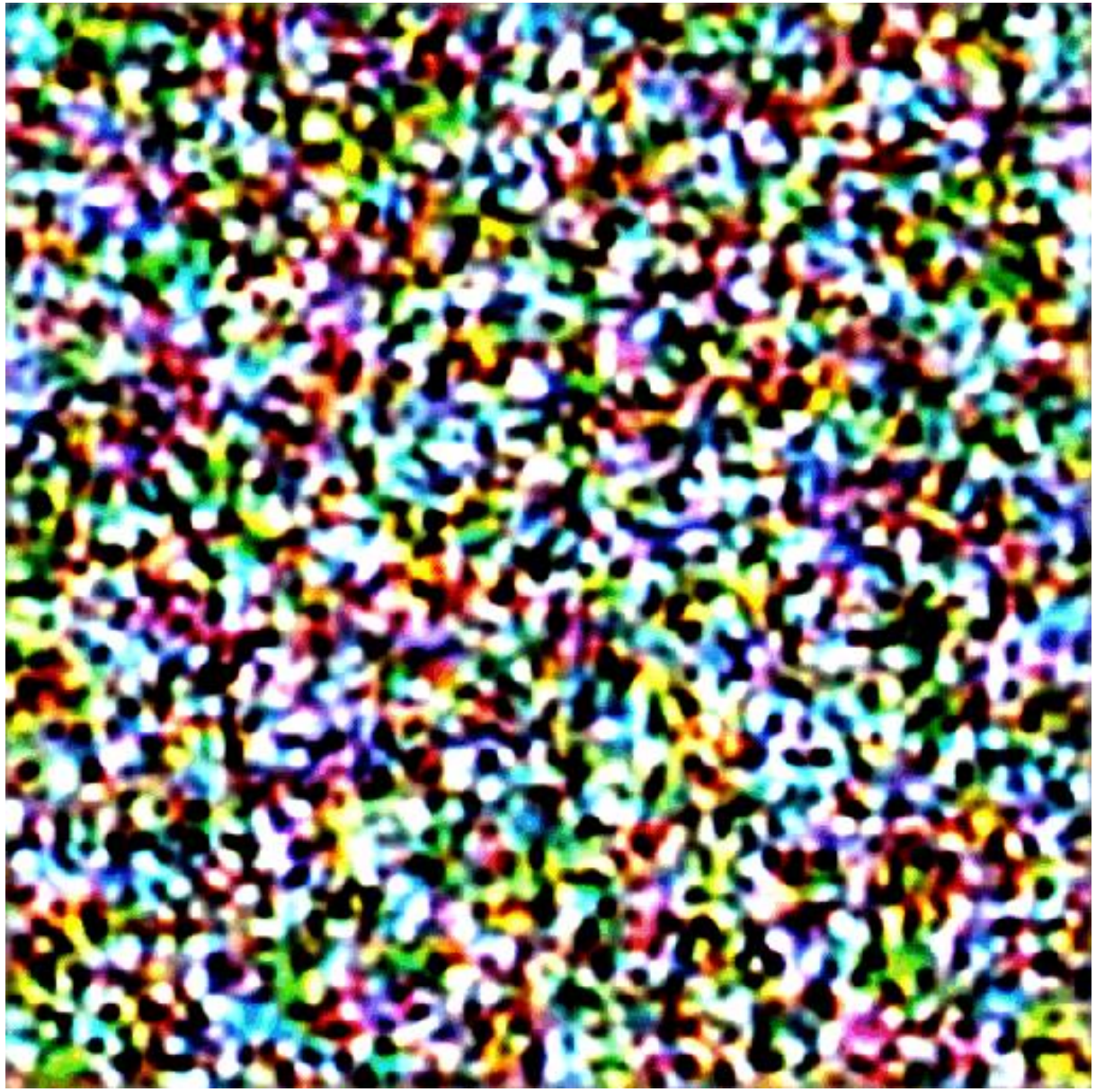


Steps 2 and 3 are repeated for a certain number of sampling steps, for example, 20 times.

Text-to-image

Step 4. Finally, the decoder of VAE converts the latent image back to pixel space. This is the image you get after running Stable Diffusion.





OK, then let’s jumping into it…

Stable Diffusion Models (checkpoint models)

Base models are AI image models trained with billions of images of diverse subjects and styles. They cost a lot of money and expertise to create, and only a few of them exist.

The most popular base models are:

- Stable Diffusion v1.5
- Stable Diffusion XL
- Flux.1 dev

The checkpoint model is not the only model type.

We also have **textual inversion** (also called embedding), **LoRA, LyCORIS, and hypernetwork**.

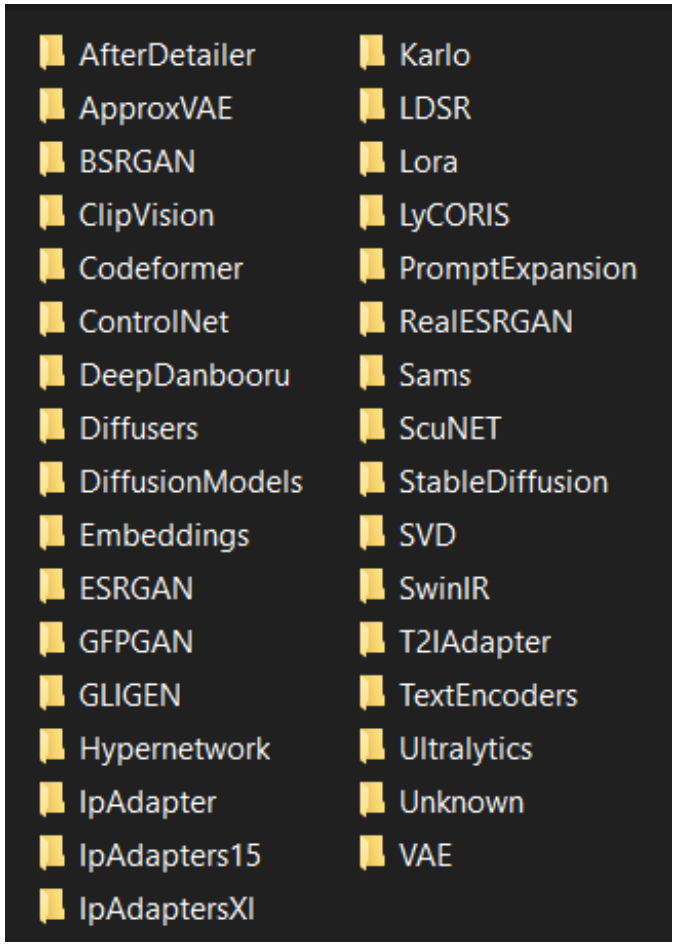
LoRA is a great way to customize AI art models without filling up local storage. So…we will now focus on this one.

LoRA applies small changes to the most critical part of Stable Diffusion models: The **cross-attention layers**. It is the part of the model where the image and the prompt meet. Researchers found it sufficient to fine-tune this part of the model to achieve good training.

Alright…open your stable diffusion app.

We are going to do simple workshop now…

Model folder



Put checkpoint model into

[StableDiffusion]

Put LoRA model into

[Lora]

Workshop #1 : Character creation

1. Create specific character with **clear background**
2. Can be **well-known character** or **specific body characteristic and specific cloth**
3. Character must be in **Full-Body view**
4. Character must be in **standing pose**
5. Create 3 images in this condition





Workshop #2 : Make a random or specific pose

1. Character must be the same character from previous workshop
2. Edit the prompt and parameter to make a various pose
3. Create 3 images in this condition (3 difference poses)





Workshop #3 : Control a camera view

1. Character must be the same character from previous workshop
2. Use either prompt editing or LoRA to Control camera view
3. Create 3 difference camera views image (difference perspective vanishing point)





Workshop #4 : Put everything together…☺

1. Character must be the same character from previous workshop
2. Give a background prompt to your image
3. You can make your character do some facial expression
4. Create a perfect 1 image with your idea…



